DOI:10.19981/j.cn23-1581/g3.2021.18.007

基于前后端分离和 Spring Security 的用户登录功能的设计与实现*

宋馨来，张海涛
（重庆工程学院，重庆 400056）

摘　要：针对使用 Spring Security 安全框架实现 Web 应用程序的认证和授权功能时存在代码量大、配置繁杂、耦合度高等问题，而 Web 应用程序初期版本只需提供简单用户登录功能的情况，提出了一种使用 SpringBoot+Spring Security 快速实现 Web 应用程序的用户认证并保留支持授权扩展的解决方案。设计了一种基于前后端分离的 Web 应用程序开发模式，并给出使用 SpringBoot+Spring Security+MyBatis 开发后台服务和使用 Vue.js+Node.js 开发前端页面的实现过程。

关键词：前后端分离；用户登录；Vue.js；Spring Security；SpringBoot；Node.js

中图分类号：TP393.09　　**文献标志码**：A　　**文章编号**：2095-2945(2021)18-0019-04

Abstract: Aiming at the issues of large amount of code, complex configuration and high coupling in development process of the Web application´s authentication and authorization by using Spring Security framework, and the initial version of the Web application only need to provide a simple user login function, this paper proposes a fast implementation of the Web application user authentication by using SpringBoot and Spring Security and keep support authorization extension solution. This paper designs a Web application development mode based on frontend-backend separation, and gives the implementation process of using SpringBoot, Spring Security and MyBatis to develop the background service and using Vue.js and Node.js to develop the front page.

Keywords: frontend-backend separation; user login; Vue.js; Spring Security; SpringBoot; Node.js

由于使用 B/S 模式开发的项目具有无须安装即可使用和升级维护方便的优点，B/S 模式逐渐超越 C/S 模式成为主流的应用程序开发模式。而使用 Java 开发 Web 应用程序又是 B/S 开发模式的主流方式。传统的 Java Web 应用程序开发基于 Servlet 技术，通常使用 JSP+JavaBean+Servlet 进行开发。这种开发模式存在以下缺点：

(1) 对项目开发人员的技能要求高，项目开发周期较长。该开发模式下，通常开发人员不仅要设计与开发后台服务，还要负责前端页面的设计与开发，如果由专门的前端开发人员来开发前端页面，最后还需要将前端代码和后端代码融合并进行功能联调，这显然会增加开发时间降低开发速度。前后端拉通则要求开发人员熟练掌握前后端开发技术，会增加开发人员的技能学习成本，给开发人员带来沉重的负担。

(2) 从项目维护和扩展的角度来说，随着网站流量的增加和项目架构的不断演进，项目势必要进行升级扩展和结构优化。传统的开发模式中前后端代码耦合在一起，造成代码的可读性以及可扩展性不高，给后期项目的维护和扩展增加了难度和成本，可能会影响项目的更新上线速度，降低网站的访问流量，对企业造成损失[1]。

前后端分离开发模式下，后台服务只负责提供数据、前端负责解析数据和页面渲染，后端开发人员负责业务和数据接口，前端开发人员负责展现和交互逻辑，前后端可以并行开发来提高开发效率，前后端分开部署则避免了职责不清晰导致的开发人员踢皮球问题，提高了问题定位效率，进而加快项目上线速度，有利于抢占市场[2]。

因前后端分离的开发模式较好地解决了传统的 Java Web 应用程序开发模式存在的对开发人员的技能要求高、开发周期较长和项目维护及扩展困难等问题，而逐渐兴起。

1 前后端分离模式

如果将 Web 页面当作前端，那么后端的功能就是提供数据，则可以将前后端分离模式简化，如图 1 所示。

前后端分别由专门的开发人员进行开发并且分开部署，分工明确，互不影响，可以准确判定问题是谁的，避免了前后端代码糅合在一起无法及时判定责任方而导致互

* 基金项目：重庆市教育委员会科学技术研究项目（项目编号：KJZD-K202001902）
作者简介：宋馨来（1977-），男，硕士研究生，讲师，研究方向：嵌入式系统、软件工程技术应用。

中国知网 https://www.cnki.net

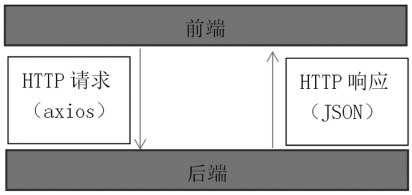

图 1 前后端分离架构示意图

相推诿的问题。

前后端交互可简化为前端通过 URL 调用后端 API 接口传递数据。当然，在软件开发之前，需要前后端的开发人员对 API 接口的访问路径、参数和返回数据的类型等协商达成一致。

2 用户登录功能的实现

后端采用 SpringBoot+Spring Security 安全框架，前端则采用目前比较流行的前端框架 Vue.js，两者结合实现前后端分离[3]。

2.1 用户登录功能的后端实现

采用 SpringBoot+Spring Security 安全框架并且前后端不分离的情况下，实现用户登录功能是比较简单的，主要工作为编写表单登录页面和在 Web Security Configuration 的 configure(HttpSecurity http)中配置表单认证，登录系统认证成功后会跳转到目标页面。在前后端分离时这种方法就行不通了，可在 configure(HttpSecurity http)中添加自定义的登录过滤器来实现，前端根据后端返回的 JSON 数据确定登录成功与否，并由前端决定登录成功后如何跳转页面。关键代码如下：

```
protected void configure (HttpSecurity http) throws Exception {
        ……
        // 使用过滤器处理登录请求
        http.addFilterAt (loginFilter (), UsernamePasswordAuthenticationFilter.class);
    }
    @Bean
    LoginFilter loginFilter() throws Exception {
        LoginFilter loginFilter = new LoginFilter();
        // 设置登录成功后给前端返回数据的回调处理
        loginFilter.setAuthenticationSuccessHandler ((request, response, authentication) -> {
            response.setContentType("application/json;charset=utf-8");
            PrintWriter out = response.getWriter();
            SystemUserDetails principal=(SystemUserDetails) authentication.getPrincipal();
            System.out.println("response:"+principal.getUsername()+"登录成功! ");
            String token=authenticationSuccessHandler
                                .onAuthenticationSuccessNew(request,response,authentication);
principal.setPassword(token);
                // 构造 JSON 数据并返回给前端
            ResponseBean msg=ResponseBean.success("登录成功! ",principal);
            String str =new ObjectMapper ().writeValueAsString(msg );
            out.write(str);
            out.flush();
            out.close();
        });
        // 设置登录失败后给前端返回数据的回调处理
        loginFilter.setAuthenticationFailureHandler ((request, response, exception) -> {
        // 构造登录失败消息并返回给前端
        });
        loginFilter.setAuthenticationManager (authenticationManagerBean());
        // 配置匹配本过滤器的 URL// 设置登录请求的 URL
        loginFilter.setFilterProcessesUrl("/login");
        return loginFilter;
    }
```

自定义的 LoginFilter 类覆写 UsernamePasswordAuthenticationFilter 处理用户登录认证的接口 attemptAuthentication。

```
public class LoginFilter extends UsernamePasswordAuthenticationFilter {
    @Override
    public Authentication attemptAuthentication(HttpServletRequest request,
                                HttpServletResponse response) throws AuthenticationException {
        //System.out.println("method:"+ request.getMethod());
        if (! request.getMethod().equals("POST")) {
            throw new AuthenticationServiceException(
                                "Authentication method not supported: " + request.getMethod());
        }
```


中国知网 https://www.cnki.net

```
//处理 JSON 格式数据的用户登录请求
        if ( request.getContentType().contains(MediaType.APPLICATION_JSON_VALUE)
        || request.getContentType().contains(MediaType.APPLICATION_JSON_UTF8_VALUE))
        {
            //获取登录请求中的用户登录数据
            Map<String,String>loginData=new HashMap<>();
            try {
                loginData = new ObjectMapper().readValue(request.getInputStream(),
    Map.class);
            } catch (IOException e) {
            }
            String username=loginData.get(getUsernameParameter());
            String password=loginData.get(getPasswordParameter());
            username=username.trim();
            // 生成一个基本的 Authentication
            UsernamePasswordAuthenticationToken authRequest =
            new UsernamePasswordAuthenticationToken(username, password);
            // 为该 Authentication 设置详细信息
            setDetails(request, authRequest);
            // 调用 ProviderManager，将该 Authentication 传给认证流程中
            // AuthenticationProvider 的 additionalAuthenticationChecks 使用
            return this.getAuthenticationManager().authenticate(authRequest);
            } else {
                return super.attemptAuthentication(request, response);
            }
        }
    }
```

2.2 用户登录功能的前端实现

前端采用 Vue.js 实现，Vue.js 是一个轻量级的构建用户界面的渐进式 JavaScript 框架，其核心库只关注视图层，不仅简单易用，还便于与第三方库或既有项目整合，与工具链以及各种支持类库结合使用时，Vue.js 也完全能够为复杂的单页应用提供驱动。

使用 Vue CLI 创建 Vue 项目，并结合 Node.js 实现了前端的部署，实现了前后端的完全分离。前端项目的关键代码如下：

项目入口代码 main.js:

```
import Vue from 'vue'
import ElementUI from 'element-ui'
// 手动引入 CSS 文件
import 'element-ui/lib/theme-chalk/index.css';
import App from './App.vue'
import router from './router'
import store from './store'
// 导入全局样式表
import './assets/styles/globe.css'
Vue.use(ElementUI)

// 配置 axios 全局使用
import axios from 'axios'
// 配置后台服务的基本 URL
axios.defaults.baseURL='http://127.0.0.1:8081/'
// 将 vue 的原型 http 对象设置为 axios 以方便调用
Vue.prototype.$http = axios

new Vue({
    router,
    store,
    render: h => h(App)
}).$mount('#app')
```

登录页面 Login.vue 中处理用户登录的方法定义如下：

```
Login() {
    // 向后端发起登录 HTTP 请求(this.loginForm 中包含用户名和密码信息)
    this. $http.post ('/login', this.loginForm).then (resp => {
        // resp 为 HTTP 响应对象，其 data 对象存储后端返回的 JSON 数据
        if (resp) {
            console.log(resp.data.obj.password)
            let path = this.$route.query.redirect;
            // 登录成功后跳转到目标页面
            this.$router.replace ((path == '/' || path
```

（下转 24 页）

中国知网 https://www.cnki.net

表 1 I/O 信号说明

信号名称	描述
di_Rbelt0FF	用于判断物料的有无
di_Rbelt0N	用于判断物料的有无
do_belt	用于控制传送带启动和停止
do_cylinder	用于控制气缸伸缩
do_sucker	用于控制真空吸盘取放

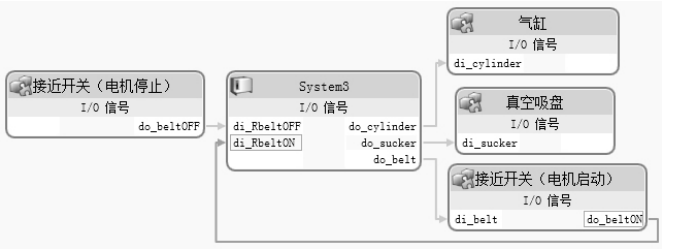

图 9 工作站逻辑设计

```
        IntiAll；  !  调用初始化程序[2]
        Start_to_Blet；!  用吸盘将物料从前道工序物料台搬运至传送带上
        Belt_Transfer；!  将物料通过传送带运至当前工序附近
        Belt_to_End；!  将物料从传送带搬运至当前工序物料台上
        walking_track；!  按物料外形走轨迹
    ENDPROC
        PROC  IntiAll()   !  初始化子程序
            VelSet  100,200；   !  定义最高速度
            Reset  do_sucker；   !  真空吸盘初始状态
            Reset  do_cylinder；!  气缸初始状态
            Reset  do_belt；!  传送带初始状态
            MoveJ  Phome,v1000,z100,ToolFrame\WObj:=wobj0;  !让机器人回到准备位置
        ENDPROC
        PROC  Belt_Transfer()  !  传送带输送子程序
            WaitDI  di_RbeltON,1; !等待接近开关触发
            WaitTime  0.2;  !  等待 0.2 秒
            Set  do_belt;    !  传送启动
            WaitTime  0.2;  !
            Movej  offs(beltpick,0,0,50),v150,fine,ToolFrame\WObj:=wobj0;  !机器人移至放置点正上方
        ENDPROC
```

4 结论

本文结合企业实际和教学，应用 RobotStudio 软件搭建了自动生产线工作站，通过 Smart 组件功能实现了工作站的仿真，动画效果直观、逼真，不仅方便教学演示，更有利于程序的调试，对生产和教学都具有应用价值。

参考文献：

[1]叶晖.工业机器人工程应用虚拟仿真教程[M].北京：机械工业出版社，2013.
[2]杨杰.工业机器人操作与编程[M].北京：机械工业出版社，2017.

（上接 21 页）

```
== undefined) ? '/home' : path);
            }else{
                // 登录失败处理
            }
        })
    }
```

对登录请求的 URL 进行路由配置后，前端就可以通过 axion 发起 HTTP 请求访问后端 API，然后根据后端返回的 JSON 数据进行前端渲染。

3 结束语

用户登录功能是 Web 应用程序的必备功能，虽然采用传统的 Web 应用程序开发技术也方便实现，但随着移动互联网的迅猛发展，后台服务在设计时不仅要考虑支持 PC 端还要考虑支持移动端。基于前后端分离和 Spring Security 来实现的用户登录功能，可方便后续支持接入移动端的用户登录功能和随着后台业务拓展带来的用户授权功能。

参考文献：

[1]杜艳美,黄晓芳.面向企业级 web 应用的前后端分离开发模式及实践[J].西南科技大学学报，2018，33(2):83-87.
[2]王建，罗政，张希，等.Web 项目前后端分离的设计与实现[J].软件工程，2020，23(4):22-24.
[3]迟殿委.一种基于前后端分离架构风格的设计与实现[J].新一代信息技术，2019(12):25-29.

中国知网 https://www.cnki.net